Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

«Встроенные системы»

Лабораторная работа №3
Программирование микроконтроллеров с операционной системой реального времени (FreeRTOS)
Игра «Pong» на стенде SDK-1.1M

Выполнили:
Козаченко Данил Александрович
Марьин Григорий Алексеевич
Группа P3312
Проверил:
Гончаров Алексей Андреевич

Санкт-Петербург 2026г.

Содержание

1 Цель работы **2**

2 Задание **2**

3 Внесённые изменения в main.c **2**
3.1 Запуск добавленной логики 3

4 Организация задач и обмена данными **3**
4.1 Задачи FreeRTOS 3
4.2 Очереди и владение состоянием 3

5 Диаграмма последовательности **5**

6 Обработка клавиатуры и общей шины **5**
6.1 Сканирование и одновременные нажатия 5
6.2 Подавление дребезга и события команд 6
6.3 Согласование опроса и вывода 6

7 Игровой алгоритм **7**
7.1 Состояния матча 7
7.2 Движение и столкновения 7

8 Добавленные экраны и управление **8**
8.1 Порядок использования 8

9 Звуковое сопровождение окончания матча **9**
9.1 Подготовка и хранение фрагмента 9
9.2 Таймеры и декодирование 9
9.3 Условия запуска и остановки 9

10 Вывод **10**

A Основные фрагменты программы **11**
A.1 Создание объектов и запуск задач 11
A.2 Обработка команд и игрового шага 11
A.3 Расчёт отбития мяча 12
A.4 Вывод аудио и связь с завершением матча 12

1 Цель работы

Реализовать в `main.c` игру «Pong» для двух игроков: обработку клавиш, движение ракеток и мяча, подсчёт очков и отображение состояния матча. Разделить опрос клавиатуры, игровую логику и вывод на экран между задачами FreeRTOS.

2 Задание

Реализовать на стенде SDK-1.1M игру с двумя ракетками и мячом. Ракетки располагаются у левой и правой границ поля. Игроки перемещают их вверх и вниз, отбивая мяч. Пропуск мяча приносит очко противнику; матч заканчивается при достижении семи очков.

Программа должна выводить легенду перед стартом, поддерживать одновременное управление двумя игроками, паузу и выход в меню. Дополнительно реализованы обратный отсчёт перед подачей, ускорение мяча после отбитий, сохранение лучшей серии отбитий в пределах сеанса работы и звуковое сопровождение победы. Использование FreeRTOS является обязательной частью реализации.

Таблица 1: Назначение клавиш

Клавиша	Действие
1 / 7	Ракетка первого игрока: вверх / вниз
3 / 9	Ракетка второго игрока: вверх / вниз
2	Старт из меню; новый матч после завершения
0	Пауза и продолжение во время игры
5	Выход в меню; прекращение звукового сопровождения

3 Внесённые изменения в main.c

В пользовательскую часть `main.c` добавлены структура игрового состояния, функции `Pong_*` и обработка действий двух игроков. Реализованы следующие изменения:

1. Добавлены структура `Pong` и функции запуска матча, перемещения ракеток, движения мяча, обработки столкновений и подсчёта очков.
2. Созданы задачи `KeyboardTask`, `GameTask`, `DisplayTask` и `LogTask`; их запуск вынесен в `Pong_InitRTOS()`.
3. Добавлены очереди `input_queue`, `frame_queue`, `log_queue` и мьютекс `i2c_mutex` для взаимодействия задач игры.
4. В `Pong_ReadKeys()` реализована маска нажатых клавиш. Добавлены независимое управление двумя ракетками, обработка удержания и выделение однократных команд старта, паузы и выхода.
5. Реализована `Pong_Draw()` с легендой, игровым полем, счётом, обратным отсчётом, паузой и экраном победителя.
6. Добавлено воспроизведение первых 15 секунд аудиозаписи после завершения матча. Фрагмент сжат до 60000 байт; выход в меню и новый матч останавливают звук.

3.1 Запуск добавленной логики

В `main()` добавлены вызовы инициализации звука и запуска задач игры. Функция `Pong_InitRTOS()` создаёт очереди, мьютекс и четыре задачи, после чего передаёт выполнение планировщику.

Листинг 1: Добавленные вызовы в main

```
Audio_Init();
Pong_InitRTOS();
```

Периодический опрос и расчёт игрового состояния выполняются внутри добавленных задач. Обычный цикл `while (1)` больше не вызывает `Calculator_Poll()`.

4 Организация задач и обмена данными

4.1 Задачи FreeRTOS

Для игры добавлены четыре задачи. В таблице 2 приведены их периоды, приоритеты и выделенные размеры стеков. Опрос клавиатуры получил приоритет 3, игровая логика — 2, отображение и журналирование — 1.

Таблица 2: Параметры прикладных задач

Задача	Приоритет	Период	Стек, байт	Назначение
`KeyboardTask`	3	10 мс	1536	Сканирование и подавление дребезга
`GameTask`	2	10 мс	2048	Состояние матча, движение, счёт
`DisplayTask`	1	40 мс	2048	Формирование и передача кадра
`LogTask`	1	По событию	1024	Вывод сообщений в USART6

Команда `vTaskDelayUntil()` сохраняет плановый период, отсчитывая его от предыдущего момента запуска. Заданы 100 Гц для ввода и игровой логики и 25 кадров/с для OLED. Фактическая частота зависит от времени выполнения операций.

4.2 Очереди и владение состоянием

Только `GameTask` изменяет структуру `Pong`. Другие задачи передают или получают копии данных, что исключает одновременное изменение положения мяча и ракеток несколькими задачами.

Таблица 3: Объекты межзадачного взаимодействия

Объект	Размер	Использование
`input_queue`	1 `InputState`	Последняя маска клавиш и время опроса
`frame_queue`	1 `Pong`	Снимок состояния для отрисовки
`log_queue`	8 × 64 байта	Сообщения о режимах и забитых мячах
`i2c_mutex`	Мьютекс	Доступ к общей шине I^2C

В `input_queue` и `frame_queue` записывается последнее состояние. Запись выполнена через `xQueueOverwrite()`, чтение — через `xQueuePeek()`. Таким образом, ракетки получают актуальную маску удержания, а экран — снимок текущего матча.

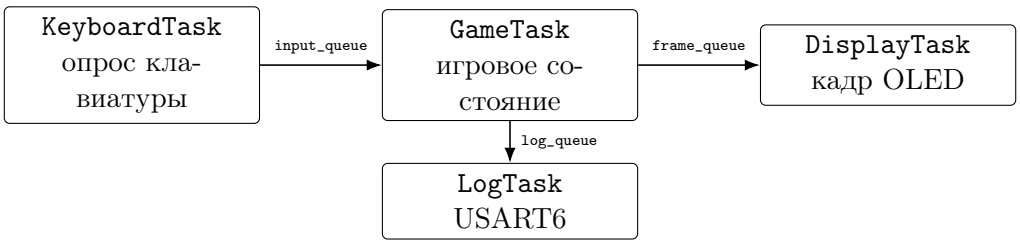


Рис. 1: Передача копий данных между задачами

5 Диаграмма последовательности

На рисунке 2 показан один цикл обмена. В реальной программе периоды задач различаются: несколько шагов игры могут приходиться на одно обновление экрана. Горизонтальные стрелки не означают прямого вызова одной задачи другой: состояния передаются через очереди.

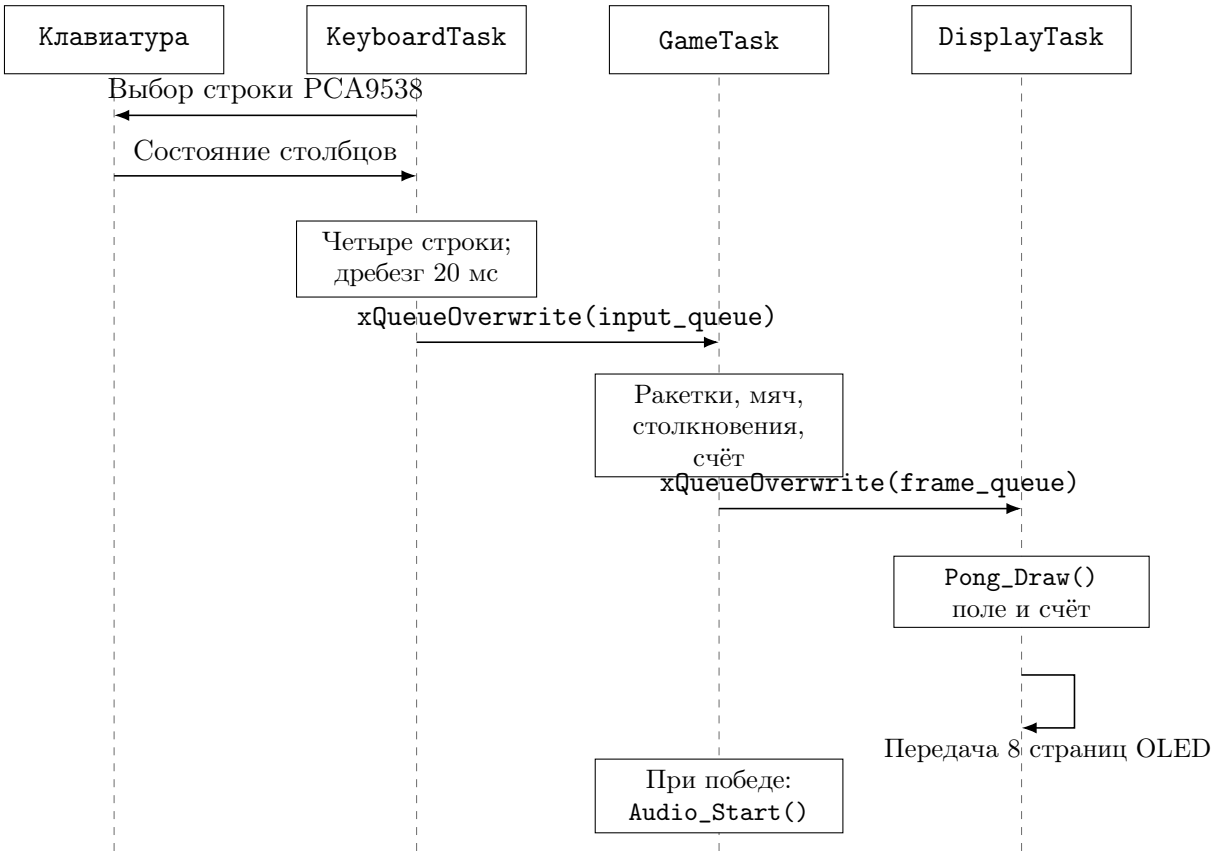


Рис. 2: Опрос клавиатуры, обработка игрового шага и вывод кадра

Мьютекс охватывает полный цикл выбора и чтения строк клавиатуры. При обновлении дисплея он захватывается отдельно для каждой страницы. Таким образом, задача экрана не удерживает шину на время передачи всего кадра, а клавиатура может получить доступ между страницами.

6 Обработка клавиатуры и общей шины

6.1 Сканирование и одновременные нажатия

Добавленная функция `Pong_ReadKeys()` возвращает маску нажатий вместо одного символа. Это позволяет одновременно удерживать по одной клавише каждого игрока. В `KeyboardTask` результат передаётся игровой задаче вместе со временем опроса.

Каждая клавиша представлена битом в 12-разрядной маске:

```
bit = 3 * row + column; mask |= 1u « bit;
```

Порядок соответствует раскладке `1 2 3 / 4 5 6 / 7 8 9 / * 0 #`. Для управления используются биты 0 и 6 у первого игрока, 2 и 8 у второго. Удержание одной клавиши

каждым игроком обрабатывается одновременно. Если у одного игрока нажаты оба направления, их действия взаимно компенсируются.

Для защиты от возможных фантомных нажатий при обнаружении более двух клавиш функция возвращает нулевую маску. Это ограничение относится и к попытке нажать командную клавишу при двух уже удерживаемых направлениях.

6.2 Подавление дребезга и события команд

Для каждого бита хранятся предполагаемое состояние и момент его изменения. Изменение принимается после 20 мс устойчивого состояния. В очередь передаются подтверждённая маска и время последнего опроса. Если снимок старше 100 мс, игровая задача считает все кнопки отпущенными, чтобы ракетка не продолжала двигаться по устаревшим данным.

Перемещение ракеток определяется удержанием, а команды запуска, паузы и выхода выполняются только по фронту нажатия:

Листинг 2: Выделение новых нажатий в GameTask

```
uint16_t pressed = input.held & ~previous;
previous = input.held;
GameMode old_mode = game.mode;
uint8_t goal = Pong_Tick(&game, input.held, pressed, now);
Audio_OnModeChange(old_mode, game.mode);
```

6.3 Согласование опроса и вывода

В `KeyboardTask` добавлен захват `i2c_mutex` на время опроса матрицы. В `DisplayTask` мьютекс захватывается отдельно для каждой страницы экрана и освобождается после её передачи. Для этого добавлена функция `oled_UpdatePage()`.

Такое разделение позволяет клавиатуре получить доступ к шине между страницами кадра. Игровая задача при этом продолжает обрабатывать своё состояние, не ожидая завершения обновления всего экрана.

7 Игровой алгоритм

7.1 Состояния матча

Структура `Pong` хранит режим, координаты ракеток и мяча, компоненты скорости, счёт, победителя, время подачи и длину серии отбитий. Переходы между основными состояниями показаны на рисунке 3.

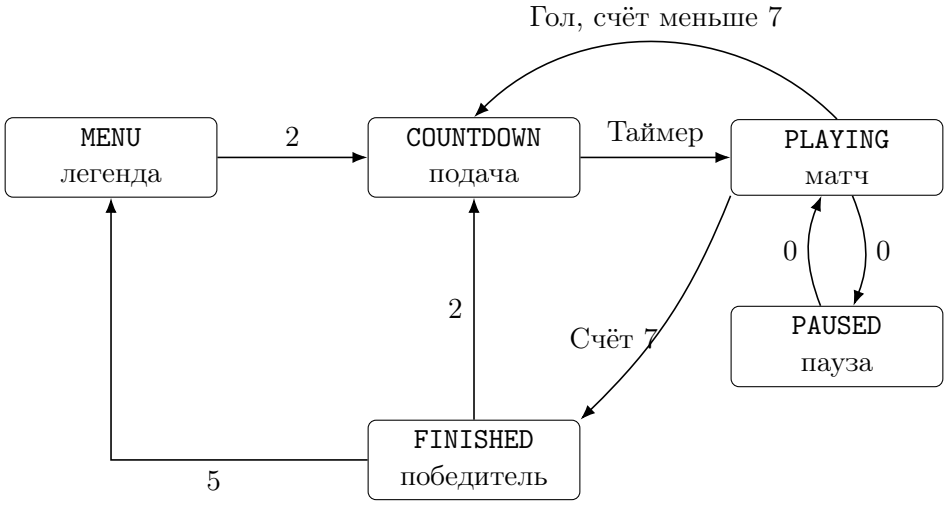


Рис. 3: Основные переходы; клавиша 5 возвращает в MENU из любого режима

Перед первой подачей выдерживается 3 с, после гола — 1,5 с. Ожидание реализовано сравнением текущего времени с `deadline`, поэтому задачи продолжают работать. На паузе координаты не изменяются. После победы показываются итоговый счёт и лучшая серия, запускается 15-секундный аудиофрагмент. Новый матч сбрасывает счёт; рекорд серии сохраняется до перезапуска программы.

7.2 Движение и столкновения

Вычисления выполняются с фиксированной точкой: одному пикселю соответствует `Q=512` единиц. При периоде 0,01 с координаты обновляются так:

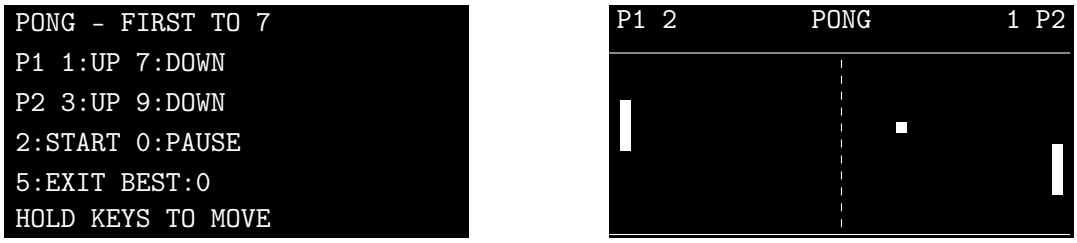
```
x += vx; y += vy;
```

Компоненты `vx`, `vy` выражены в единицах фиксированной точки за шаг, а не в пикселях в секунду. Начальная горизонтальная скорость равна 230 / (512 × 0,01) ≈ 44,9 пикселя/с. После каждого отбития её модуль увеличивается на 22 единицы, но ограничивается значением 650 (около 127 пикселей/с). Ракетки перемещаются на 840 единиц за шаг, то есть примерно 164 пикселя/с.

При ударе о верхнюю или нижнюю границу вертикальная скорость меняет знак. Столкновение с ракеткой определяется по пересечению её плоскости за шаг. Высота пересечения вычисляется интерполяцией между старой и новой позициями, поэтому мяч не пропускает ракетку при увеличении скорости. Вертикальная скорость после отбития зависит от расстояния до центра ракетки; почти горизонтальные траектории получают минимальную вертикальную составляющую. Выход за левую или правую границу даёт очко противнику. Фрагмент расчёта отбития приведён в приложении А.

8 Добавленные экраны и управление

В функции `Pong_Draw()` добавлены отдельные варианты отрисовки для меню, подачи, игры, паузы и окончания матча. На поле выводятся две ракетки, мяч, горизонтальные границы, пунктирная средняя линия и счёт игроков. Размер ракетки выбран 3×14, мяча — 3×3 пикселя. В меню отображается легенда, а после победы — итоговый счёт и лучшая серия отбитий.



а) Главное меню

б) Пример игрового поля

Рис. 4: Схематическое представление экранов, составленное по коду отрисовки

Передача кадра разделена на восемь страниц по 128 байт. `DisplayTask` захватывает шину для одной страницы, отправляет её, освобождает мьютекс и переходит к следующей. Состояние для кадра является локальной копией: изменения игры во время передачи не меняют уже сформированный буфер.

8.1 Порядок использования

1. После включения прочитать легенду. Клавиша 2 запускает новый матч.
2. Во время обратного отсчёта можно установить ракетки в начальное положение.
3. Удерживать 1/7 для левой ракетки и 3/9 для правой. Нажатия разных игроков допускаются одновременно.
4. Клавиша 0 останавливает движение; повторное нажатие продолжает матч.
5. При семи очках появляется экран победителя. Нажать 2 для нового матча или 5 для возврата к легенде; оба действия останавливают музыку.

Счёт и рекорд не записываются в энергонезависимую память. Возврат в меню не завершает работу микроконтроллера: программа продолжает опрашивать клавиатуру и обновлять экран.

9 Звуковое сопровождение окончания матча

9.1 Подготовка и хранение фрагмента

Для экрана победы использованы первые 15 с предоставленной записи Morgenshtern «Новый Мерин». Скрипт `tools/prepare_audio.py` декодирует фрагмент, смешивает каналы в моно, фильтрует высокие частоты перед уменьшением частоты дискретизации и кодирует результат в IMA ADPCM. По краям фрагмента предусмотрено короткое плавное изменение амплитуды. Исходный MP3 не хранится в прошивке.

Каждый отсчёт представлен четырьмя битами; в байте находятся два кода, сначала младший, затем старший. Начальные значения предиктора и индекса адаптивного шага равны нулю. Размер аудиоданных:

$$15 \times 8000 = 120000 \text{ сэмплов;}$$
$$120000 \times 4 \,/\, 8 = 60000 \text{ байт.}$$

Это примерно 58,6 КиБ; для несжатого 16-разрядного моносигнала той же длительности потребовалось бы 240000 байт. Массив `audio_clip` объявлен как `const` и размещается во Flash, без копирования всей записи в оперативную память.

9.2 Таймеры и декодирование

TIM2 формирует ШИМ на PA5, подключённом к излучателю звука. TIM6 вызывает обработчик выборок с частотой 8 кГц. При тактовой частоте таймеров APB1 84 МГц и `PSC=0`:

$$\text{Частота ШИМ: } 84000000 \,/\, (2099 + 1) = 40000 \text{ Гц;}$$
$$\text{частота выборок: } 84000000 \,/\, (10499 + 1) = 8000 \text{ Гц.}$$

В прерывании выбирается очередной полубайт, восстанавливается знаковый 16-разрядный отсчёт `pcm` и изменяется коэффициент заполнения:

```
CCR1 = ((pcm + 32768) * 2100) » 16;
```

Декодер хранит только предыдущее значение сигнала и индекс таблицы шага. Полный PCM-буфер не создаётся. Прерывание имеет приоритет 6, не вызывает функции FreeRTOS и не выполняет операции I^2C.

9.3 Условия запуска и остановки

`Audio_OnModeChange()` запускает звук только при переходе в `FINISHED`. Повторные шаги в том же состоянии не перезапускают запись. После обработки 120000 отсчётов обработчик отключает таймер выборок и звуковой выход. Выход в меню и запуск нового матча прекращают запись досрочно. Изменение состояния проигрывателя из задачи защищено критической секцией. Качество ограничено форматом 8 кГц и характеристиками штатного излучателя; он используется как дополнительное сопровождение игры.

10 Вывод

В пользовательской части `main.c` реализована игра «Pong» для двух игроков. Добавлены движение ракеток и мяча, обработка столкновений, подсчёт очков до семи, обратный отсчёт перед подачей, пауза и выход в меню. Управление назначено на клавиши 1/7 и 3/9; перед стартом выводится легенда.

Опрос клавиш, игровая логика, формирование кадра и вывод сообщений распределены между добавленными задачами. Для обмена состоянием использованы очереди, для согласования опроса и вывода — мьютекс.

После завершения матча проигрывается 15-секундный аудиофрагмент. Для уменьшения объёма записи добавлены подготовка моносигнала 8 кГц, кодирование IMA ADPCM и декодирование при воспроизведении. Размер аудиоданных составляет 60000 байт. Запуск нового матча и возврат в меню прекращают воспроизведение.

A Основные фрагменты программы

Фрагменты соответствуют пользовательской части `main.c`. Таблица аудиоданных в приложение не включена: её содержимое генерируется скриптом подготовки записи и не меняет алгоритм проигрывания.

A.1 Создание объектов и запуск задач

Листинг 3: Функция Pong_InitRTOS

```
static void Pong_InitRTOS(void)
{
  input_queue = xQueueCreate(1, sizeof(InputState));
  frame_queue = xQueueCreate(1, sizeof(Pong));
  log_queue = xQueueCreate(8, 64);
  i2c_mutex = xSemaphoreCreateMutex();
  if (!input_queue || !frame_queue || !log_queue || !i2c_mutex)
    Error_Handler();
  if (xTaskCreate(KeyboardTask, "Keyboard", 384, NULL, 3,
                  &keyboard_task) != pdPASS ||
      xTaskCreate(GameTask, "Pong", 512, NULL, 2,
                  &game_task) != pdPASS ||
      xTaskCreate(DisplayTask, "OLED", 512, NULL, 1,
                  &display_task) != pdPASS ||
      xTaskCreate(LogTask, "UART", 256, NULL, 1, NULL) != pdPASS)
    Error_Handler();
  Pong_Log("PONG FreeRTOS: 1/7=P1 3/9=P2 2=start 0=pause 5=menu");
  vTaskStartScheduler();
  Error_Handler();
}
```

A.2 Обработка команд и игрового шага

Листинг 4: Основные переходы Pong_Tick

```
static uint8_t Pong_Tick(Pong *g, uint16_t held,
                         uint16_t pressed, uint32_t now)
{
  if (pressed & KEY_5) { g->mode = MENU; return 0; }
  if (g->mode == MENU || g->mode == FINISHED) {
    if (pressed & KEY_2) Pong_Start(g, now);
    return 0;
  }
  if (pressed & KEY_0) {
    if (g->mode == PLAYING) g->mode = PAUSED;
    else if (g->mode == PAUSED) g->mode = PLAYING;
    return 0;
  }
  if (g->mode == PAUSED) return 0;
  Pong_MovePaddles(g, held);
  if (g->mode == COUNTDOWN) {
    if ((int32_t)(now - g->deadline) >= 0) {
      g->mode = PLAYING; g->countdown = 0;
    } else {
      g->countdown = (uint8_t)((g->deadline - now + 999u) / 1000u);
    }
    return 0;
  }
  return Pong_Physics(g, now);
}
```

A.3 Расчёт отбития мяча

Листинг 5: Изменение скорости при попадании в ракетку

```
static void Pong_Bounce(Pong *g, int32_t paddle_y, int direction)
{
  int32_t speed = g->vx < 0 ? -g->vx : g->vx;
  int32_t offset = g->ball_y - (paddle_y + (PADDLE_HEIGHT / 2) * Q);
  speed = Clamp(speed + 22, 230, 650);
  g->vx = direction * speed;
  g->vy = Clamp(offset * speed / (8 * Q), -500, 500);
  if (g->vy > -65 && g->vy < 65) g->vy = offset < 0 ? -65 : 65;
  ++g->rally;
  if (g->rally > g->best_rally) g->best_rally = g->rally;
}
```

A.4 Вывод аудио и связь с завершением матча

Листинг 6: Обработчик таймера выборок

```
void TIM6_DAC_IRQHandler(void)
{
  if (!(TIM6->SR & TIM_SR_UIF)) return;
  TIM6->SR = 0;
  if (!audio_playing) return;
  uint32_t position = audio_sample_pos;
  if (position >= AUDIO_SAMPLE_COUNT) {
    Audio_Disable();
    ++audio_completed;
    return;
  }
  uint8_t packed = audio_clip[position / 2];
  uint8_t code = (position & 1u) ? packed >> 4 : packed & 15u;
  int16_t pcm = ADPCM_Decode(&audio_decoder, code);
  TIM2->CCR1 = ((uint32_t)((int32_t)pcm + 32768)
                * audio_pwm_period) >> 16;
  audio_sample_pos = position + 1;
}
```

Листинг 7: Однократный запуск и досрочное прекращение записи

```
static void Audio_OnModeChange(GameMode before, GameMode after)
{
  if (before == after) return;
  if (after == FINISHED) Audio_Start();
  else if (before == FINISHED || after == MENU) Audio_Stop();
}
```